SIGN LANGUAGE TO SMART-GLASSES HUD: REAL-TIME ISL WORD DISPLAY ON AN ESP32-C3 OLED HEAD-UP DISPLAY

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF

B.TECH DEGREE

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

BY

[STUDENT NAME]

Roll No.: [ROLL NUMBER]

Under the Guidance of

[FACULTY ADVISOR]



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

[MONTH, YEAR]

DECLARATION

I hereby declare that the internship report entitled **"Sign Language to Smart-Glasses HUD: Real-Time ISL Word Display on an ESP32-C3 OLED Head-Up Display"** is my original work carried out during the internship at **Department of ECE, IIIT Senapati, Manipur (in-house project)**. The work presented in this report has been completed by me under the guidance of the **supervisor/mentor concerned and faculty advisor.** All sources of information, ideas, figures, and other material obtained from external sources have been appropriately acknowledged and cited.

I further declare that the information and results presented in this report are true to the best of my knowledge and that the report has not been submitted elsewhere for the award of any other degree or qualification.

Student Name: _____

Roll No.: _____

Date: _____

Signature: _____

CERTIFICATE

This is to certify that the internship report entitled **"Sign Language to Smart-Glasses HUD: Real-Time ISL Word Display on an ESP32-C3 OLED Head-Up Display"**, submitted to the Department of Electronics and Communication Engineering, Indian Institute of Information Technology Senapati, Manipur, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering, is a bonafide record of the work carried out by **[STUDENT NAME]**, bearing **Roll No. [ROLL NUMBER]**, during the internship at **Department of ECE, IIIT Senapati, Manipur (in-house project).**

Signature of Faculty Advisor

(Faculty Advisor/Mentor)

Examiner 1: _____ **Examiner 2:** _____

Examiner 3: _____ **Examiner 4:** _____

INTERNSHIP DOCUMENTS

Attach the following documents in this section:

- Internship Offer Letter
- Internship Completion Certificate
- Industry Supervisor/Mentor Certificate (if issued)
- Any other relevant supporting document

Note: this work was an in-house project at the Department of ECE, IIIT Senapati, Manipur. Attach the project approval and completion documents issued by the department here.

ABSTRACT

People who use Indian Sign Language (ISL) often cannot be understood by people who do not sign. This project builds a low-cost, wearable aid for the listener: a pair of glasses with a small head-up display (HUD) that shows, in real time, the words a signer is making. The work was carried out as an in-house project at the Department of Electronics and Communication Engineering, IIIT Senapati, Manipur. A laptop camera films the signer. A recognition application, written by my project partner, extracts hand and body landmarks with MediaPipe, classifies each sign with a temporal convolutional network (TCN) and builds a sentence. Each word is sent as a small JSON packet over UDP on a phone Wi-Fi hotspot. My part of the work was the complete glasses side. An ESP32-C3 SuperMini microcontroller joins the hotspot, registers the mDNS name `glasses.local`, receives the packets on port 4210 and draws the text on a 0.96-inch SSD1306 OLED. A short word is shown large in the centre, and a sentence scrolls across the screen twice. The OLED image passes through a 25 mm lens and is reflected into the eye by a 45-degree clear acrylic sheet, so the text floats in front of the wearer. The unit runs from a 600 mAh LiPo cell through a TP4056 protection board. A parametric OpenSCAD enclosure that clips onto the glasses arm was designed and exported for 3D printing. All parts were verified with a step-by-step bring-up procedure. The link was tested end to end, with typed text and with simulated and live signing. The bench optics gave a sharp image at 22 mm, and the measured sentence timing matched the model. The project gave hands-on experience in embedded firmware, wireless networking, optics, power electronics and mechanical CAD.

ACKNOWLEDGEMENT

I sincerely thank my faculty advisor, [FACULTY ADVISOR], for guidance and encouragement throughout this project. I thank the Department of Electronics and Communication Engineering and the Indian Institute of Information Technology Senapati, Manipur, for the laboratory facilities, including the bench power supply and the digital multimeter used for testing. I thank my project partner for building the sign-recognition software and for working with me on the shared interface. Finally, I thank my family and friends for their constant support.

[STUDENT NAME]

TABLE OF CONTENTS

1. Introduction 10
2. Technical Background / Literature Survey 12
3. Work Performed and Methodology 15
4. Results and Discussion 21
5. Challenges and Problem-Solving 26
6. Learning Outcomes 28
7. Conclusion and Future Scope 29

References 30

Appendices/Annexures 31

LIST OF FIGURES

Figure 1.1 System overview: from the signer to the text in the wearer's eye 10

Figure 2.1 HUD light path (side view): OLED → lens → 45° acrylic → eye 13

Figure 3.1 Overall project workflow 15
Figure 3.2 Wiring of the glasses display unit 17
Figure 3.3 Firmware flowchart of the glasses receiver 18
Figure 3.4 3D model of the HUD display unit (OpenSCAD render) 19
Figure 3.5 Cross-section of the enclosure: OLED rails at the top, lens plate, 45° acrylic in the head 19

Figure 4.1 Sentence scroll time vs. sentence length (2 passes at 60 px/s) 22
Figure 4.2 Printable enclosure parts (OpenSCAD renders) 23
Figure 4.3 All four parts laid out on one print bed (137 × 161 mm) 23

LIST OF TABLES

Table 2.1 Existing ways to show sign-language output to a listener 13

Table 3.1 Hardware used for the display unit 16
Table 3.2 Software and tools used 16
Table 3.3 Phase-wise work summary 20

Table 4.1 Hardware bring-up test results 21
Table 4.2 Measured electrical values (Rigol DM3058E) 22
Table 4.3 Enclosure parts: plastic and print time (slicer estimates) 24
Table 4.4 Expected (design) vs. obtained results 25

Table 5.1 Problems and the actions taken 27

Table A.1 Display unit wiring 31

Chapter 1
Introduction

1.1 Background

- Indian Sign Language (ISL) is the main language of the Deaf community in India.
- Most hearing people do not know ISL, so everyday conversations need a human interpreter.
- Computer vision can now track hands and body in real time on an ordinary laptop camera.
- Small, cheap Wi-Fi microcontrollers and OLED screens make a home-built head-up display (HUD) possible.
- Combining the two gives a "caption" for sign language that the listener sees in their glasses.

1.2 Internship Overview

- **Organization:** Department of ECE, IIIT Senapati, Manipur (in-house project).
- **Type:** combined CS + ECE semester project, done by two students.
- **My role:** hardware and firmware: ESP32-C3 + OLED display unit, power, optics, enclosure and the glasses-side link.
- **Partner's role:** the sign-recognition software (dataset, model, training, live app) [12].
- **Shared contract:** one UDP packet format, agreed by both (Section 3.4).
- **Duration of the work logged here:** 24 October 2026 to 1 November 2026.

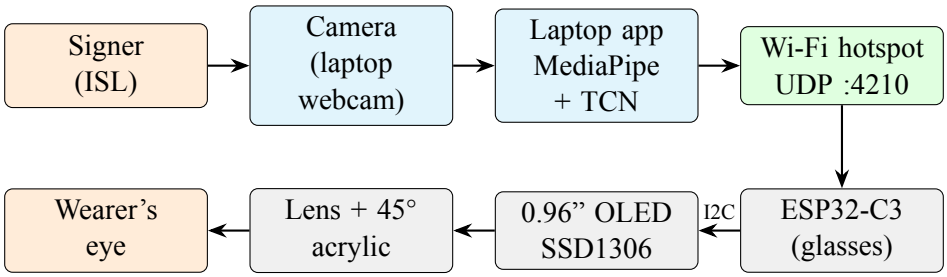


Figure 1.1: System overview: from the signer to the text in the wearer's eye

1.3 Problem Statement / Technical Problem

- Show each recognised ISL word to a hearing person **in their field of view**, hands-free, in real time.
- The display unit must be **light, battery powered and wireless**, and must clip onto ordinary glasses.
- The text must be **sharp and readable** through simple optics (one lens and a beam combiner).
- The glasses must work with the recognizer app over a **simple, documented network link**.

1.4 Motivation

- Phone-screen captions force the listener to look away from the signer.
- Commercial smart glasses are expensive and closed.
- A low-cost DIY HUD built from hobby parts can be made and repaired by students.
- Earlier DIY HUD designs used weak electronics (no regulator, ESP8266-01, hard-coded IP) [11]; this project fixes those.

1.5 Objectives

1. Bring up and verify every hardware part (ESP32-C3, OLED, charger, battery, switch) step by step.
2. Write firmware that receives words over Wi-Fi (UDP + mDNS) and shows them clearly on the OLED.
3. Design the HUD optics: OLED → lens → 45° acrylic → eye.
4. Design a 3D-printable clip-on enclosure for the display unit.
5. Connect the glasses to the recognizer app and test the full chain end to end.

1.6 Scope of the Internship

- **In scope:** isolated ISL words (INCLUDE-50 [5] and own recorded words) and short sentences built from them; display unit for one eye (right); laptop as the recognition computer.
- **Out of scope:** continuous sentence-level sign translation; recognition running on the glasses.
- **Stretch goal:** an ESP32-CAM on the glasses as the camera.

Chapter 2

Technical Background / Literature Survey

2.1 Relevant Concepts

- **ESP32-C3:** 32-bit RISC-V microcontroller with Wi-Fi 802.11 b/g/n (2.4 GHz only) and native USB [1].
- **SSD1306 OLED:** 128×64 monochrome display driver, controlled over I2C (address 0x3C) [2].
- **I2C bus:** two-wire serial bus (SDA data, SCL clock) between the microcontroller and the OLED.
- **UDP:** connectionless transport; one small datagram per word gives low delay and no connection set-up.
- **mDNS:** lets the laptop find the glasses by name (`glasses.local`) instead of a changing IP address.
- **TP4056:** single-cell Li-ion linear charger (CC/CV, 4.2 V) with a protection IC [3].
- **HUD optics:** a convex lens placed closer to the OLED than its focal length makes a magnified *virtual* image; a 45° clear sheet reflects it into the eye while the world stays visible (Figure 2.1).

Thin-lens equation (real-is-positive sign convention) [7]:

$$\frac{1}{u} + \frac{1}{v} = \frac{1}{f}, \qquad m = -\frac{v}{u} \tag{2.1}$$

where u is the object (OLED) distance, v the image distance and f the focal length. If $u < f$, then $v < 0$: the image is virtual, upright and magnified.

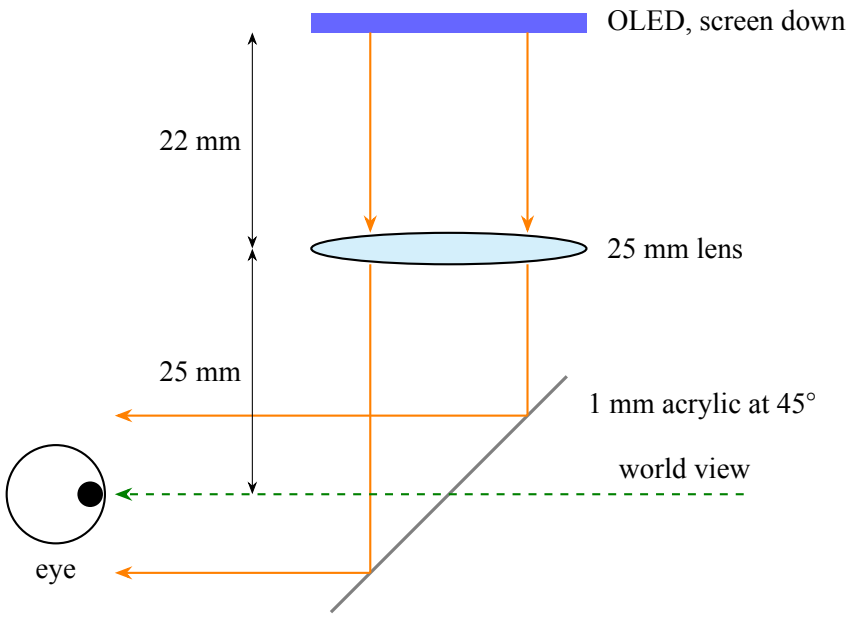


Figure 2.1: HUD light path (side view): OLED → lens → 45° acrylic → eye

2.2 Existing Technologies / Methods

Table 2.1: Existing ways to show sign-language output to a listener

Method	How it works	Limitation
Human interpreter	Person translates live	Not always available, costly
Phone / laptop captions	Recognised text on a screen	Listener looks away from the signer
Commercial smart glasses	Built-in micro display	Expensive, closed platform
Sensor gloves	Flex/IMU sensors on the signer's hands	Signer must wear special hardware
This project	Camera + ML on laptop, DIY HUD glasses	Needs a laptop and a hotspot

2.3 Literature Survey

- **MediaPipe** [4]: real-time pipelines for hand and body landmark tracking on ordinary CPUs and GPUs; used by the recognizer to turn video into landmark sequences.
- **INCLUDE dataset** [5]: a large ISL word dataset (263 words, 4,287 videos); its 50-word subset INCLUDE-50 is used for training.

- **Temporal Convolutional Networks** [6]: 1-D dilated convolutions that match or beat recurrent networks on sequence tasks; the recognizer's main model is a TCN.
- **June_Glasses** [11]: open DIY HUD glasses; gave the optics concept (OLED, lens, beam combiner). Its electronics had no regulator and a hard-coded IP.
- **Adafruit SSD1306 / GFX libraries** [8, 9]: standard Arduino drivers for the OLED, with text, bitmaps and size scaling.
- **OpenSCAD** [10]: script-based CAD; every dimension is a named parameter, so the enclosure can be changed and re-exported quickly.

2.4 Identified Gap / Technical Requirement

- No low-cost, open, clip-on HUD that shows **ISL** recognition output was found.
- **Requirements derived for this project:**
  - Wireless link with discovery by name (mDNS) and a fixed, small packet format.
  - Readable text: large, centred words; long sentences scroll.
  - Safe single-cell LiPo power with protection and a power switch.
  - Adjustable focus and a mount that fits a given pair of glasses.
  - Software-selectable mirroring (`MIRROR_X`, `FLIP_Y`) in case the optics flip the text.

Chapter 3

Work Performed and Methodology

3.1 Overall Workflow

- The work followed a "test every part first" approach: each step has a clear pass/fail check.
- Hardware was tested on a breadboard with friction-fit headers; soldering is done only in the final build.

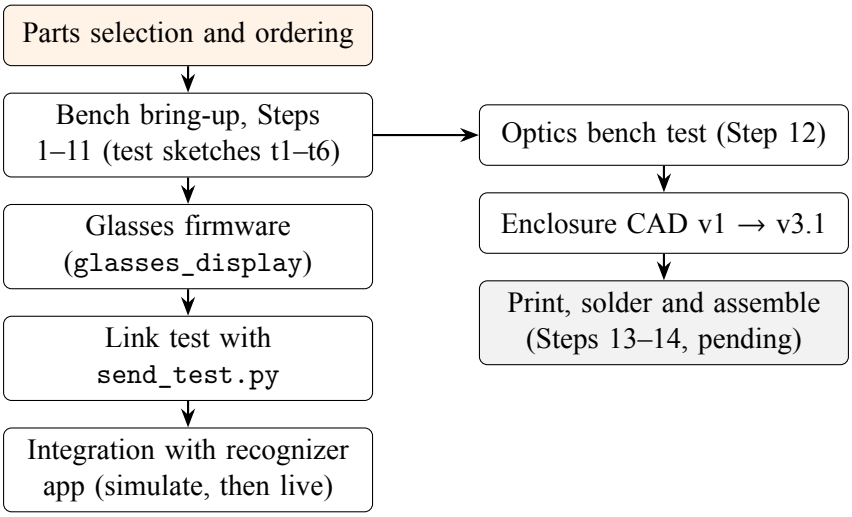


Figure 3.1: Overall project workflow

3.2 Materials/Hardware/Software/Tools Used

Table 3.1: Hardware used for the display unit

Part	Qty	Purpose / note
ESP32-C3 SuperMini	1 (+1)	Wi-Fi microcontroller, native USB-C
0.96” SSD1306 OLED, 128×64, I2C	1 (+1)	Display, address 0x3C
LiPo 3.7 V, 600 mAh (602030)	1	Battery, ~30×20×6 mm
TP4056 Type-C board with protection	1	Charger + over-discharge protection
Slide switch	1	Power on/off
25 mm lens (marked f45)	1	Magnifies the OLED image
Clear acrylic, 1 mm, 30×30 mm	1	45° beam combiner
PLA filament (~73 g)	–	3D-printed enclosure
Breadboard, berg strips, jumpers, multimeter	–	Bench bring-up
Lab bench supply, Rigol DM3058E meter	–	Charging and measurements

Table 3.2: Software and tools used

Tool	Use
Arduino IDE 2.3, ESP32 core 3.3.12	Firmware build and upload
Adafruit SSD1306 + GFX, WiFiUdp, ESPmDNS	OLED drawing, UDP, mDNS
Python 3 + PyYAML	`send_test.py` link-test tool
Python, PyTorch, MediaPipe, OpenCV	Recognizer app (partner’s software)
OpenSCAD	Parametric 3D enclosure, STL export
Git	Version control and daily handoff log

3.3 Design and Development Methodology

Electronics and wiring

- LiPo → TP4056 (B+/B−) → OUT+ → slide switch → ESP32-C3 `5V`; OUT− → `G`.
- ESP32-C3 → OLED: `3V3`→VDD, `G`→GND, `GPIO8`→SDA, `GPIO9`→SCK (SCL).
- The ESP32-C3's on-board regulator gives 3.3 V from the 3.7–4.2 V cell.
- **Safety:** the stock TP4056 charges at 1 A (1.7C for 600 mAh), so its USB port is taped ("NO USB"); the cell is charged only from a bench supply at 4.20 V / 0.30 A.

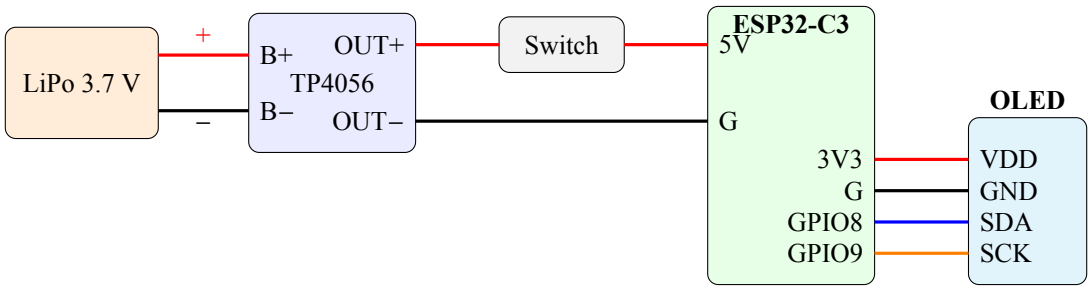


Figure 3.2: Wiring of the glasses display unit

Firmware design (`glasses_display.ino`)

- **Boot:** start I2C and the OLED, apply `MIRROR_X`/`FLIP_Y`, show `WIFI...`.
- **Network:** join the hotspot, lower Wi-Fi TX power to 8.5 dBm (the SuperMini antenna is weak), start mDNS `glasses` and UDP port 4210, show `READY` + IP.
- **Receive:** read one datagram into a 256-byte buffer and pull the `"text"` value out of the JSON (a non-JSON packet is shown raw).
- **Display rule:** text at size 2 (12×16 px per character) that fits 128 px (≤10 characters) is centred; longer text scrolls right-to-left, 2 passes at 60 px/s, then the screen goes blank.
- Scroll position is computed from elapsed time, so the speed stays constant however long a redraw takes.

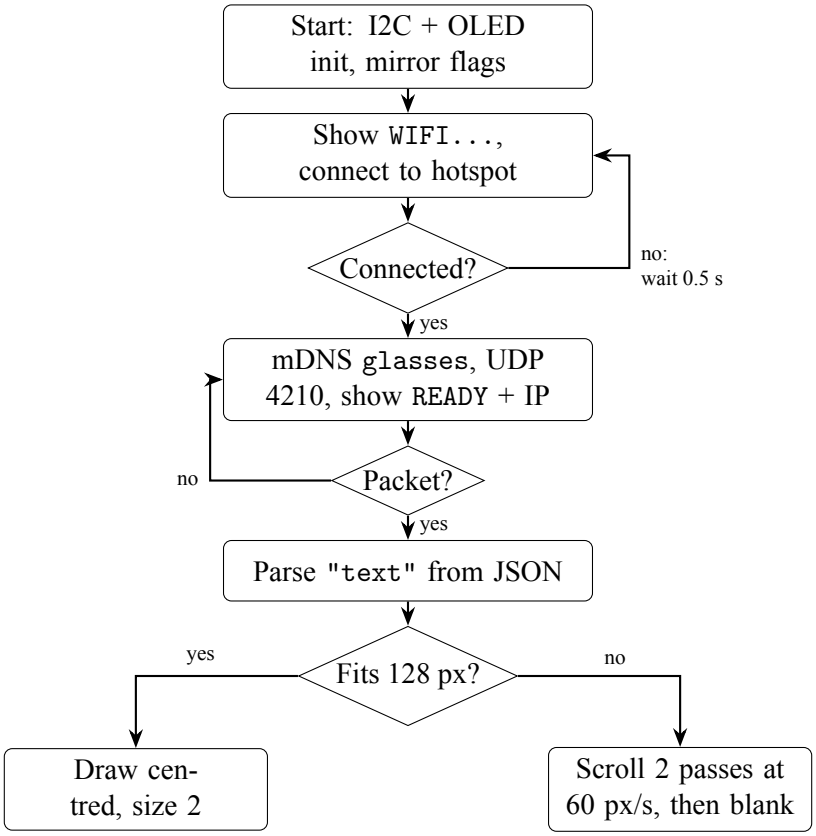


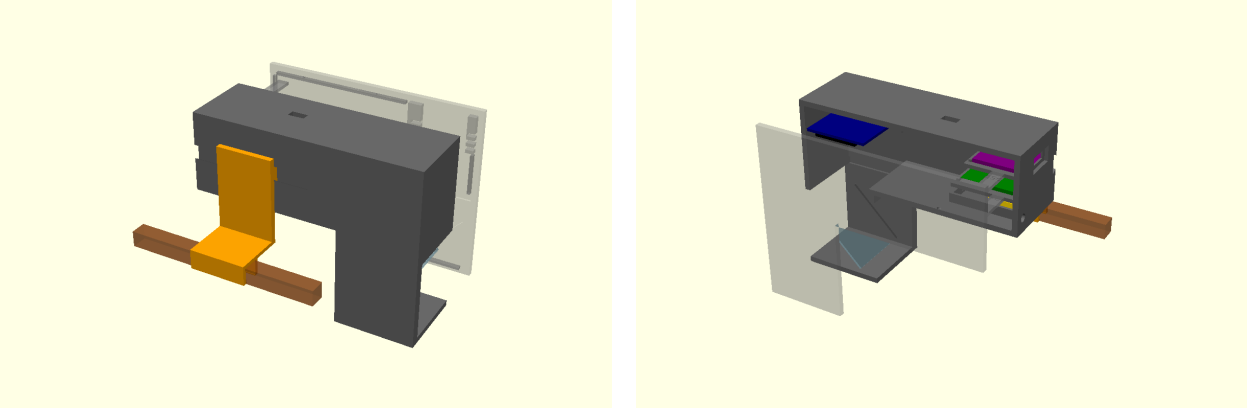
Figure 3.3: Firmware flowchart of the glasses receiver

Optics design

- First design used a mirror and $u = 40$ mm, based on the lens marking $f = 45$ mm.
- The bench test (Step 12) showed a sharp image at only $u = 22$ mm, so the mirror was removed.
- Final path: OLED (screen down) → 22 mm → lens → 25 mm → 45° acrylic → eye (Figure 2.1).
- The OLED sits in rail slots at $u = 19$, 22 or 25 mm, so focus can be adjusted after printing.

Enclosure design (OpenSCAD, v3.1)

- Fully parametric: optics distances, clearances, board sizes and glasses-arm sizes are named values.
- A pocket for each part (LiPo, TP4056, ESP32-C3, switch); the lid clicks shut and its fingers hold the boards.
- Clip-on mount: a dovetail slot in the body plus a swappable clip for the 7×7 mm temple arm.
- The build script stops with an error if any parts collide; the clip snap strain is checked (2.2%, below PLA's ~2.5% limit).



(a) Assembly on the glasses arm (temple side) (b) Inside view (lid see-through)

Figure 3.4: 3D model of the HUD display unit (OpenSCAD render)

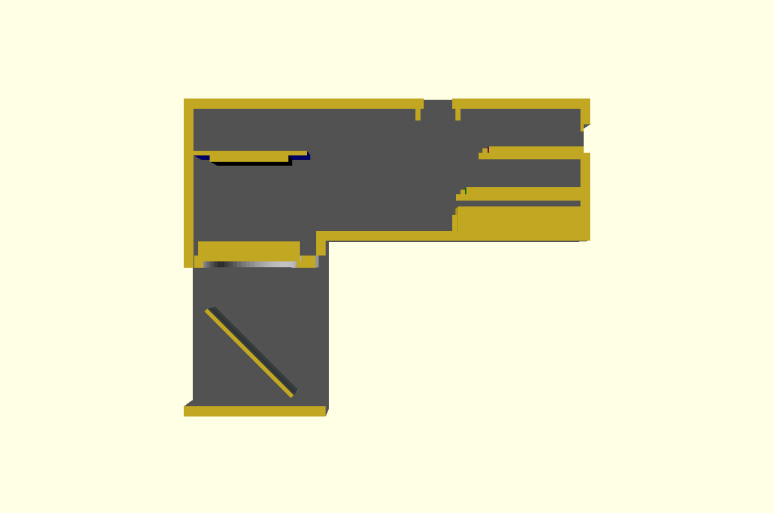


Figure 3.5: Cross-section of the enclosure: OLED rails at the top, lens plate, 45° acrylic in the head

Recognizer software (project partner's work)

- Webcam → MediaPipe hand + body landmarks [4] → TCN classifier [6] (BiLSTM and Transformer also available) → sentence builder → optional English translation.
- Trained on INCLUDE-50 [5] and own recorded signs; runs on the Mac's GPU.
- `glasses.py` sends each word, then the finished sentence, to the glasses over UDP [12].
- A `--simulate` mode sends fake sentences, so the glasses can be tested without a camera or model.

3.4 Experimental/Simulation/Implementation Procedure

1. **Step 1:** visual check; measure the board's 5V and 3V3 pins with the multimeter.
2. **Steps 2–3:** board on friction-fit headers; `t1_blink` (LED + Serial `alive N`).
3. **Step 4:** `t2_i2c_scan` (expect 0x3C), `t3_oled_hello`, mirror check.
4. **Steps 5–6:** `t4_wifi_udp`; then on the laptop:
   `python tools/send_test.py --host <IP> --text hello`.
5. **Steps 7–9:** TP4056 and switch on USB only; bench-charge the LiPo; solder the battery leads.
6. **Steps 10–11:** run the ESP32-C3 from the battery; repeat the tests on the spare boards.
7. **Step 12:** optics bench: slide the OLED until the text through the lens is sharpest; add the acrylic at 45°.
8. **Integration:** flash `glasses_display`; run `python software/run.py --simulate`, then the live camera.

Packet format (one UDP datagram per word or sentence, UTF-8 JSON, under 256 bytes):

```
{"text": "hello", "conf": 0.92}
```

3.5 Week-wise or Phase-wise Work Summary

Table 3.3: Phase-wise work summary

Date	Work done
24 Oct 2026	Parts received; bring-up guide and test sketches written; Step 1 passed on both boards
25 Oct 2026	Steps 3–6: blink, I2C scan (0x3C), OLED text, Wi-Fi + UDP text from the laptop
29 Oct 2026	Enclosure v1 in OpenSCAD; LiPo checked; ESP32-C3 + OLED running on the battery
30 Oct 2026	Enclosure v2: component pockets, click lid, dovetail clip mount
31 Oct 2026	Steps 7–11 done; optics bench (sharp at 22 mm); enclosure v3/v3.1 print-ready; interface contract
1 Nov 2026	Final receiver firmware; recognizer app integrated; simulated and live runs; documentation

Chapter 4

Results and Discussion

4.1 Results

- All bring-up tests, Steps 1–11, passed on both sets of boards.
- Typed text from the laptop appears on the OLED within about 0.5 s (by eye), both by IP address and by `glasses.local`.
- The whole chain works: simulated and live signing in front of the camera show words, then the sentence, on the OLED.
- Final firmware compiles to 79% of the ESP32-C3 flash.
- Enclosure v3.1 is print-ready: all parts render with no collisions.

4.2 Experimental/Simulation/Testing Results

Table 4.1: Hardware bring-up test results

Part	Test	Result
ESP32-C3 #1, #2	5V/3V3 voltages, blink + Serial	Pass
OLED #1, #2	I2C 0x3C, text, mirror view	Pass
Wi-Fi + UDP	`send_test.py` text by IP and by `glasses.local`	Pass
TP4056 #1, #2	B+ ≈ 4.2 V on USB (no cell)	Pass
Slide switch ×3	Continuity swaps when flipped	Pass
Bench supply	4.20 V / 0.30 A set and checked with the meter	Pass
LiPo #1, #2	Voltage, soldered leads, “NO USB” tape	Pass
Battery power	`READY` + words on battery, switch on/off	Pass
Battery life	Runtime on a full cell	Done (value not logged)
Optics	Sharp text through lens + acrylic	In progress
Soldering, enclosure	Final build	Pending

Table 4.2: Measured electrical values (Rigol DM3058E)

Quantity	**Measured**
ESP32-C3 5V pin (USB)	5.162 V
ESP32-C3 3V3 pin (USB)	3.302 V
LiPo #1 as received	3.999 V (~80%)
ESP32-C3 5V pin on battery, under load	3.967 V
Best focus, OLED to lens (u)	22 mm

4.3 Graphs, Tables and Figures

Sentence scroll time. At text size 2 each character is 12 px wide; one pass moves the text across its own width plus the 128 px screen:

$$t_{\text{pass}} = \frac{12\,n + 128}{60}\ \text{s}, \qquad t_{\text{total}} = 2\,t_{\text{pass}} \tag{4.1}$$

where n is the number of characters and 60 px/s is `SCROLL_PX_PER_S`.

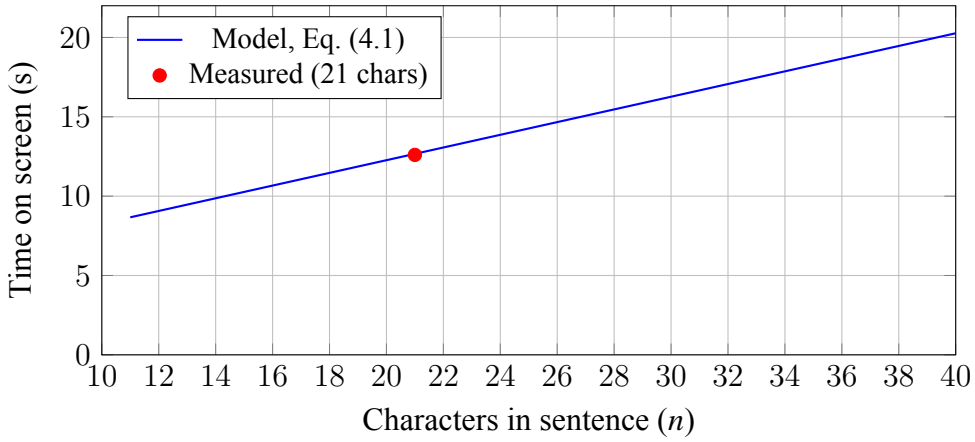
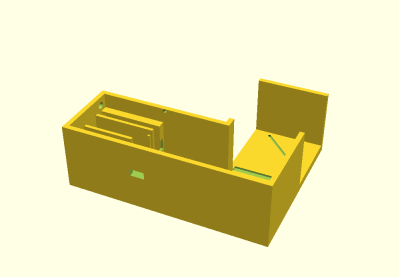
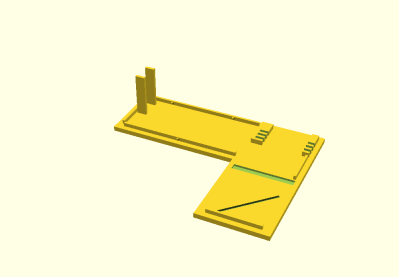


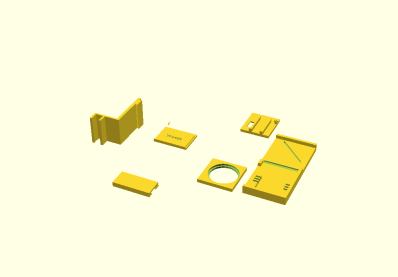
Figure 4.1: Sentence scroll time vs. sentence length (2 passes at 60 px/s)



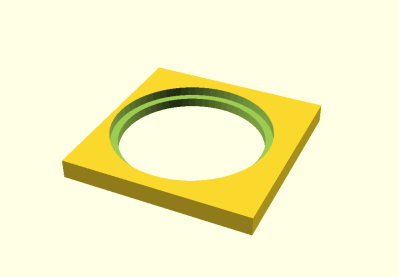
(a) Body



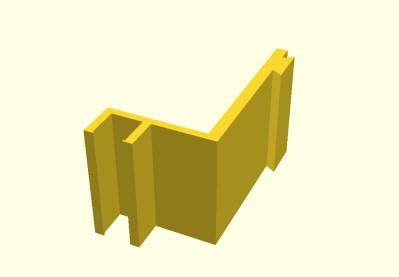
(b) Lid



(c) Test-fit piece



(d) Lens plate



(e) Clip

Figure 4.2: Printable enclosure parts (OpenSCAD renders)

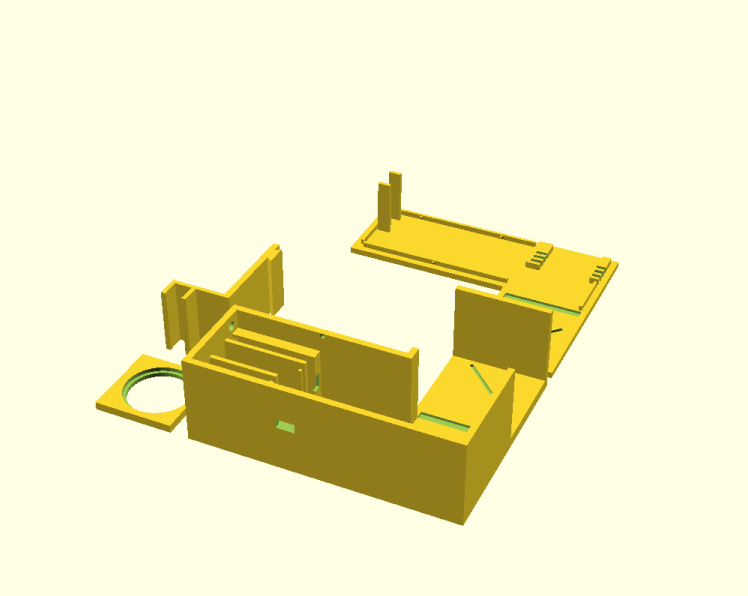


Figure 4.3: All four parts laid out on one print bed (137 × 161 mm)

Table 4.3: Enclosure parts: plastic and print time (slicer estimates)

Part	PLA (g)	Time (normal / fast printer)
Body	50	4.2 h / 1.7 h
Lid	16	1.3 h / 0.5 h
Clip	5	0.4 h / 0.2 h
Lens plate	2	0.1 h
All on one bed	**~73**	**~6 h / ~2.5 h**
Test-fit piece (optional)	27	2.2 h / 0.9 h

4.4 Analysis and Discussion

- **Power:** 5.162 V and 3.302 V are inside the expected 4.8–5.2 V and 3.2–3.4 V ranges. On battery the 5V pin reads the cell voltage (3.967 V) and 3V3 stays regulated, so the board runs safely on one LiPo cell.
- **Display timing:** for "I am going to school." ($n = 21$), Eq. (4.1) gives $2 \times (12 \cdot 21 + 128)/60 = 12.67$ s; adding the 2 s word gap of the simulator gives 14.67 s. The measured gap was **14.6 s**, so the firmware and the app stay in step (Figure 4.1).
- **Optics:** with $u = 22$ mm and an estimated $f \approx 25$ mm, Eq. (2.1) gives

$$\frac{1}{v} = \frac{1}{25} - \frac{1}{22} = \frac{22 - 25}{550} = -\frac{3}{550} \;\Rightarrow\; v \approx -183 \text{ mm}, \quad m = -\frac{v}{u} = \frac{183}{22} \approx 8.3$$

  So the text appears as an upright, about 8× magnified virtual image roughly 18 cm in front of the lens. The value of f is an estimate; measuring it directly (Step 12, O5) is still pending.
- **Link:** UDP with one packet per word keeps the protocol simple; the sender trims text so the packet always fits the 256-byte firmware buffer.

4.5 Comparison with Expected/Reference Results

Table 4.4: Expected (design) vs. obtained results

Item	Expected	Obtained
Focus distance u	40 mm (lens marked $f = 45$ mm; Eq. (2.1) gives $v = -360$ mm, $m = 9$)	22 mm; real $f \approx 25$ mm; mirror removed
Sentence time ($n = 21$)	12.67 s scroll + 2 s gap = 14.67 s	14.6 s measured
3V3 rail	3.2–3.4 V	3.302 V
5V rail on USB	4.8–5.2 V	5.162 V
I2C address	0x3C	0x3C on both OLEDs
mDNS discovery	`glasses.local` resolves	Works; sometimes slow, IP fallback used
Clip snap strain	below PLA limit (~2.5%)	2.2% (model check)

Chapter 5

Challenges and Problem-Solving

5.1 Technical Challenges

- The lens did not behave like its marking ($f = 45$ mm): the image was sharp at 22 mm, not 40 mm.
- The ESP32-C3 SuperMini often failed to connect at full Wi-Fi transmit power.
- The ESP32-C3 cannot join 5 GHz hotspots.
- `glasses.local` (mDNS) was sometimes not found from the laptop.
- A 4-line word-wrap made sentences cramped and hard to read on a 128×64 screen.
- The simulator first sent ISL glosses ("TODAY HOT"), so words like "is" never appeared.

5.2 Practical/Implementation Challenges

- The stock TP4056 charges at 1 A, which is too high for a 600 mAh cell.
- Arduino's build tool (`ctags`) is x86-only and failed on the Apple Silicon laptop ("bad CPU type in executable").
- Port 8000 on the laptop was already in use by another application.
- Loose friction-fit headers gave random resets and "no I2C device" errors.
- Board and lens sizes were not all measured before the enclosure design.

5.3 Solutions and Corrective Actions

Table 5.1: Problems and the actions taken

Problem	Action taken
Lens focus at 22 mm, not 40 mm	Mirror removed; OLED rails at 19/22/25 mm; enclosure v3
Wi-Fi fails at full TX power	`WiFi.setTxPower(WIFI_POWER_8_5dBm)`
5 GHz hotspot	2.4 GHz hotspot ("Maximize Compatibility" on iPhone)
mDNS not always found	IP shown on the OLED; `--glasses <IP>` / `--host <IP>` option
Cramped sentences	One-line, right-to-left scroll, 2 passes, then blank
Simulator showed glosses	Simulator sends the English sentence word by word
1 A charge current	USB port taped; bench supply 4.20 V / 0.30 A, stop at ≤ 0.05 A
`ctags` "bad CPU type"	Installed Rosetta 2
Port 8000 busy	App started with `--port 9000`
Loose headers	Press the board down first; solder only in the final build
Unmeasured part sizes	0.3 mm pocket clearance; print the test-fit piece first

Chapter 6

Learning Outcomes

6.1 Technical Skills Acquired

- Embedded firmware for the ESP32-C3: I2C display driving, Wi-Fi station mode, UDP sockets, mDNS.
- Simple, robust protocol design: a fixed JSON packet with a size limit and a shared written contract.
- Thin-lens optics in practice: focusing, virtual image, magnification, beam combiner.
- Li-ion power safety: CC/CV charging, protection boards, correct charge current.

6.2 Software/Hardware Skills

- Arduino IDE / arduino-cli, Adafruit GFX/SSD1306 libraries, Serial debugging.
- Multimeter and bench supply use: voltage, continuity, current-limited charging.
- Soldering practice and breadboard prototyping.
- Parametric CAD in OpenSCAD and preparing STL files for 3D printing.
- Python scripting (`send_test.py`), Git version control, LaTeX documentation.

6.3 Professional and Communication Skills

- Working in a two-person team with a clear split of work and one shared interface.
- Writing step-by-step guides with pass/fail checks, and keeping a daily work log.
- Recording problems and their fixes so they are not repeated.

6.4 Industry Exposure

- A product-style process: requirements, bring-up, integration, then mechanical design.
- Design for manufacture: a printable part layout, clearances, and a test-fit piece before the full print.
- Use of open-source tools and libraries that are common in industry.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

- A working, low-cost chain from an ISL signer to text in a wearable HUD was built and tested.
- The glasses side (ESP32-C3, OLED, LiPo power, firmware, optics and enclosure design) is complete up to final assembly.
- Measured values match the design model, e.g. sentence timing within 0.1 s.

7.2 Key Achievements

- All hardware bring-up tests (Steps 1–11) passed.
- Wireless display firmware: name-based discovery, centred words, scrolling sentences.
- End-to-end demo: camera → recognizer → Wi-Fi → glasses.
- Print-ready parametric clip-on enclosure (~73 g PLA).
- Complete documentation: bring-up guide, run guide, interface contract and a public code repository.

7.3 Future Scope / Recommendations

- Show “NO LINK” after 10 s without packets, and reconnect to Wi-Fi automatically.
- Measure the glasses-side latency (packet in → OLED update) and the end-to-end delay.
- Measure the lens focal length directly (Step 12, O5), and check readability in daylight.
- Print, solder and assemble the final unit (Steps 13–14), and measure the battery runtime.
- Report the recognition accuracy of the model on a held-out test set.
- Add the ESP32-CAM on the glasses as the camera, so the laptop camera is not needed.

REFERENCES

[1] Espressif Systems, “ESP32-C3 Series Datasheet,” Espressif Systems, Shanghai, China. [Online]. Available: https://www.espressif.com

[2] Solomon Systech, “SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller,” datasheet, Rev. 1.1, 2008.

[3] NanJing Top Power ASIC Corp., “TP4056: 1A Standalone Linear Li-ion Battery Charger with Thermal Regulation in SOP-8,” datasheet.

[4] C. Lugaresi *et al.*, “MediaPipe: A framework for building perception pipelines,” arXiv:1906.08172, 2019.

[5] A. Sridhar, R. G. Ganesan, P. Kumar, and M. Khapra, “INCLUDE: A large scale dataset for Indian Sign Language recognition,” in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 1366–1375.

[6] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” arXiv:1803.01271, 2018.

[7] E. Hecht, *Optics*, 5th ed. Boston, MA, USA: Pearson, 2017.

[8] Adafruit Industries, “Adafruit_SSD1306 Arduino library,” GitHub. [Online]. Available: https://github.com/adafruit/Adafruit_SSD1306

[9] Adafruit Industries, “Adafruit-GFX-Library,” GitHub. [Online]. Available: https://github.com/adafruit/Adafruit-GFX-Library

[10] OpenSCAD, “OpenSCAD: The programmers solid 3D CAD modeller.” [Online]. Available: https://openscad.org

[11] perseus784, “June_Glasses,” GitHub. [Online]. Available: https://github.com/perseus784/June_Glasses

[12] integral-830, “sign-inference-bridge: ISL recognition application,” GitHub. [Online]. Available: https://github.com/integral-830/sign-inference-bridge

APPENDICES / ANNEXURES

Appendix A: Wiring table

Table A.1: Display unit wiring

From	To
LiPo red / black	TP4056 B+ / B− (soldered)
TP4056 OUT+	Switch middle pin; switch side pin → ESP32-C3 5V
TP4056 OUT−	ESP32-C3 G
ESP32-C3 3V3	OLED VDD (VCC)
ESP32-C3 G	OLED GND
ESP32-C3 GPIO8	OLED SDA
ESP32-C3 GPIO9	OLED SCK (SCL)

Appendix B: UDP interface

- Glasses: mDNS name `glasses.local`, UDP port 4210 (from `config.yaml`).
- One datagram per word or sentence, UTF-8 JSON, under 256 bytes.
- `text`: word or short line to show; `conf`: model confidence, 0–1, 2 decimals.
- Test without the ML app: `python tools/send_test.py --text hello`

Appendix C: Firmware excerpt (`glasses_display.ino`)

```
// New text: fits on the screen -> static in the centre; wider -> start scrolling.
void showText(const String &text) {
  int16_t x1, y1;
  uint16_t w, h;
  display.setTextSize(2);
  display.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
  if (w <= display.width()) {
    scrollText = "";
    display.clearDisplay();
    drawCentered(text, 2, (display.height() - 16) / 2);
    display.display();
  } else {
    scrollText = text;
    scrollW = w;
```

```
    scrollStart = millis();
    lastFrame = 0;
  }
}

void loop() {
  int len = udp.parsePacket();
  if (len > 0) {
    len = udp.read(packet, sizeof(packet) - 1);
    packet[len] = '\0';
    showText(jsonText(String(packet)));
  }
  if (scrollText.length() && millis() - lastFrame >= FRAME_MS) {
    lastFrame = millis();
    scrollFrame();
  }
}
```

Appendix D: Bring-up test sketches

- `t1_blink`: board alive (LED + Serial).
- `t2_i2c_scan`: finds the OLED at 0x3C.
- `t3_oled_hello`: big "HELLO" + counter.
- `t4_wifi_udp`: text from the laptop over Wi-Fi.
- `t5_icons`: bitmap image on the OLED.
- `t6_mirror_center`: mirrored, centred text for the HUD view.

Appendix E: Source code and guides

- Repository: `https://github.com/Charan20510/sign_lang_hud_glasses`
- Bring-up guide: `docs/hardware_bringup.md`; run guide: `docs/run_guide.md`.
- Enclosure model and print guide: `hardware/enclosure/`.